2024-11-19-MaxFlowMinCut

Generated by Scribe.AI

December 31, 2024

Slide 1

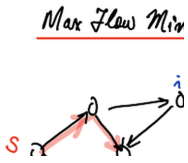
Content

Max Flow Min Cut

Description

Slide 1 of 17 in a Linear Programming course introduces the Max Flow Min Cut theorem. The slide presents a directed graph representing a network flow problem. Node S is the source, and node t is the sink. The arcs represent the flow capacity between nodes, with u_{ij} representing the capacity of the arc from node i to node j. The highlighted arcs in red likely represent a current flow assignment. The question "What is the max. flow from S to t subject to $0 \leq x_{ij} \leq u_{ij}$?" poses the core problem of finding the maximum flow from the source to the sink, given the capacity constraints on each arc. The x_{ij} represents the flow on arc (i, j). The context of linear programming is crucial because the max-flow problem can be formulated as a linear program, and the Max Flow Min Cut theorem provides a powerful tool for solving it efficiently. The slide sets the stage for the course by introducing a key problem in network optimization that will be addressed using linear programming techniques.

Figures



(a) A handwritten diagram showing a directed graph with nodes labeled S (source), t (sink), and several intermediate nodes. Edges have arrows indicating direction and some edges are highlighted in red. The text "Wij" is written above one of the edges. The question "What is the max. flow from S to t subject to $0 \leq x_{ij} \leq u_{ij}$?" is written below the graph.

Slide 2

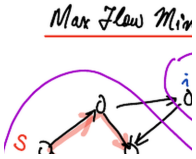
Content

Max Flow Min Cut

Description

Slide 2 of 17 in a Linear Programming course introduces the Max Flow Min Cut theorem through a graphical representation. A directed graph depicts a network flow problem, with S representing the source and t the sink. Arcs have capacities denoted by w_{ij}, representing the maximum flow allowed from node i to node j. Some arcs are highlighted in red, possibly indicating a current flow assignment. The graph is partitioned into two sets, C and C^c, by a cut, visually represented by a purple line. The source node S belongs to C, and the sink node t belongs to C^c. The capacity of the cut, denoted by $K(C)$, is defined as the sum of the capacities of the arcs going from C to C^c: $K(C) = \sum_{i \in C, j \notin C} w_{ij}$. This sets the stage for explaining the Max Flow Min Cut theorem, which states that the maximum flow from source to sink equals the minimum capacity of any cut separating the source and sink. The context of linear programming is crucial because the max-flow problem can be formulated and solved as a linear program, and the Max Flow Min Cut theorem provides an alternative, often more efficient, solution method.

Figures



(a) A handwritten diagram showing a directed graph with nodes labeled S (source), t (sink), and several intermediate nodes. Edges have arrows indicating direction and some edges are highlighted in red. The text "w_{ij}" is written above one of the edges. Two subsets of nodes are enclosed in purple ovals. One subset contains the source node S, and the other contains the sink node t. An arrow labeled "a cut" points to the boundary between the two subsets. Below the graph, the equation $K(C) = \sum_{i \in C, j \notin C} w_{ij}$ is written, along with the text "capacity of the cut C".

Slide 3

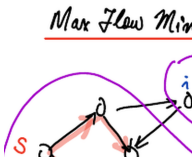
Content

Max Flow Min Cut

Description

Slide 3 of 17 in a Linear Programming course illustrates the Max Flow Min Cut theorem. The slide shows a directed graph representing a network flow problem, with S as the source and t as the sink. Arcs have capacities denoted by w_{ij}. A cut is visually represented by a purple line partitioning the nodes into two sets, C (containing S) and C^c (containing t). The arcs crossing the cut from C to C^c represent the capacity of the cut, K(C). The key inequality, "(any) Flow($s \to t$) $\leq$ (any) K(C)", is presented. This inequality states that any flow from source s to sink t is less than or equal to the capacity of any cut separating s and t. This is a crucial step in proving the Max Flow Min Cut theorem, which states that the maximum flow equals the minimum cut capacity. The red highlighted arcs likely represent a flow assignment. The context of linear programming is important because the max-flow problem can be formulated and solved as a linear program, and the Max Flow Min Cut theorem provides an alternative, often more efficient, solution method. The slide visually demonstrates a key concept leading to the theorem's proof.

Figures



(a) A handwritten diagram showing a directed graph with nodes labeled S (source), t (sink), and several intermediate nodes. Edges have arrows indicating direction and some edges are highlighted in red. The text "w_{ij}" is written above one of the edges. Two subsets of nodes are enclosed in purple ovals. One subset contains the source node S, and the other contains the sink node t. An arrow labeled "a cut" points to the boundary between the two subsets. Below the graph, the inequality "(any) Flow($s \to t$) $\leq$ (any) K(C)" is written.

Slide 4

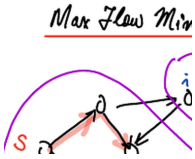
Content

Max Flow Min Cut

Description

Slide 4 of 17 in a Linear Programming course continues the explanation of the Max Flow Min Cut theorem. The slide presents a directed graph with source node S and sink node t. The arcs have capacities, with w_{ij} representing the capacity of the arc from node i to node j. A cut is shown, dividing the graph into two sets, C (containing S) and C^c (containing t). The arcs crossing the cut from C to C^c determine the capacity of the cut, $K(C)$. The highlighted arcs in salmon/light red likely represent a flow assignment. The key inequality, Flow$(s \to t) \leq K(C)$, is presented, with the addition of "any" before Flow and K(C) to emphasize that this holds for any flow and any cut. Crucially, "max" is written in red below the left "any" and "min" in red below the right "any", indicating that the maximum flow from s to t is equal to the minimum capacity of any cut separating s and t. This is the statement of the Max Flow Min Cut theorem. The context of linear programming is important because this theorem provides an efficient way to solve the max-flow problem, which can be formulated as a linear program. The slide visually reinforces the theorem's core concept.

Figures



(a) A handwritten diagram showing a directed graph with nodes labeled S (source), t (sink), and several intermediate nodes. Edges have arrows indicating direction and some edges are highlighted in salmon/light red. The text "w_{ij}" is written above one of the edges. Two subsets of nodes are enclosed in purple ovals. One subset contains the source node S, and the other contains the sink node t. An arrow labeled "a Cut" points to the boundary between the two subsets. Below the graph, the inequality (any) Flow($s \rightarrow t$) $\leq$ (any)$K(C)$ is written, with "max" written in red below "any" on the left side and "min" written in red below "any" on the right side, indicating that the inequality becomes an equality when considering the maximum flow and minimum cut.

Slide 5

Content

Table 22.1 The Travelers' Example:
Seat Availability

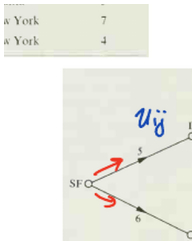
From	To	Number of seats
San Francisco	Denver	5
San Francisco	Houston	6
Denver	Atlanta	4
Denver	Chicago	2
Houston	Atlanta	5
Atlanta	New York	7
Chicago	New York	4

Description

Slide 5 of 17 in a Linear Programming course presents a network flow problem disguised as a "Travelers' Example." Table 22.1 provides the capacity of each flight route (number of seats available) between different cities: San Francisco (SF), Denver (D), Houston (H), Atlanta (A), Chicago (C), and New York City (NYC). The table serves as input data for the network flow problem visualized in Figure 22.1. Figure 22.1 is a directed graph where nodes represent cities and edges represent flight routes with capacities given by the numbers on the edges. The notation u_{ij} is written above the edge from SF to D, indicating the capacity of that edge. Red arrows on some edges suggest a possible flow assignment, illustrating a feasible solution to the network flow problem. The context of linear programming is crucial here because the problem of finding the maximum number of travelers that can be transported from SF to NYC can be formulated and solved as a linear program. The Max Flow Min Cut theorem, discussed in previous slides, could be used to solve this problem efficiently.

Figures

(a) Table 22.1 showing the seat availability for different routes in a travel network.



(b) A directed graph representing a network flow problem. Nodes represent cities (SF, D, H, A, C, NYC), and edges represent flight routes with capacities indicated by numbers on the edges. Red arrows indicate a possible flow assignment.

Slide 6

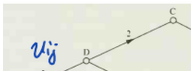
Content

Figure 22.1

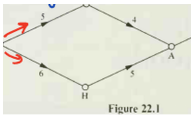
Description

Slide 6 of 17 in a Linear Programming course shows two figures (Figure 22.1 and Figure 22.2) illustrating different flow assignments in a network flow problem. Both figures depict a directed graph representing a network with nodes representing cities (SF, D, H, A, C, NYC) and edges representing flight routes with capacities indicated by numbers on the edges. Figure 22.1 shows a possible flow assignment with values v_{ij} indicated by handwritten arrows. Figure 22.2 shows a different flow assignment with values x_{ij} indicated by handwritten arrows. The text "Flow = 8" is handwritten next to Figure 22.2, suggesting the total flow in that assignment is 8. The page number "[C]p.368" is handwritten, likely referencing a textbook or other source material. The context of linear programming is crucial because these figures illustrate different feasible solutions to a network flow problem, which can be formulated and solved as a linear program. The difference between the two figures likely demonstrates different solution approaches or iterations in solving the problem.

Figures

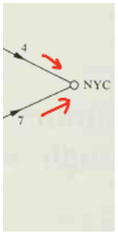


(a) A directed graph showing a network flow problem. Nodes represent cities (SF, D, H, A, C, NYC), and edges represent flight routes with capacities indicated by numbers on the edges. Arrows indicate direction. A handwritten v_{ij} is above an edge, and red arrows indicate a possible flow assignment.



(b) A second directed graph showing a network flow problem. Nodes represent cities (SF, D, H, A, C, NYC), and edges represent flight routes with capacities indicated by numbers on the edges. Arrows indicate direction. A handwritten x_{ij} is above an edge, and red arrows indicate a possible flow assignment. The text "Flow = 8" is handwritten to the left.

(c) Handwritten text "Flow = 8".



(d) A second directed graph showing a network flow problem. Nodes represent cities (SF, D, H, A, C, NYC), and edges represent flight routes with capacities indicated by numbers on the edges. Arrows indicate direction. A handwritten x_{ij} is above an edge, and red arrows indicate a possible flow assignment. The text "Figure 22.2" is below the graph.

(e) Handwritten text "[C]p.368".

Slide 7

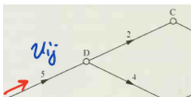
Content

Figure 22.1

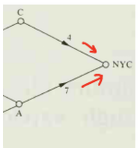
Description

Slide 7 of 17 in a Linear Programming course presents two figures (Figure 22.1 and Figure 22.2, though only Figure 22.1 is explicitly labeled) illustrating different flow assignments in the same network flow problem. Both figures depict a directed graph representing a network with nodes representing cities (SF, Denver, Houston, Atlanta, Chicago, NYC) and edges representing flight routes with capacities indicated by numbers on the edges. Figure 22.1 shows a possible flow assignment with values v_{ij} indicated by handwritten arrows. Figure 22.2 shows a different flow assignment with values indicated by handwritten red numbers on the edges. The text "Flow = 9" is handwritten next to Figure 22.2, suggesting the total flow in that assignment is 9. A purple oval encloses a subset of nodes in Figure 22.2, possibly highlighting a cut in the network. The page number "[C]p.368" is handwritten, likely referencing a textbook or other source material. The context of linear programming is crucial because these figures illustrate different feasible solutions to a network flow problem, which can be formulated and solved as a linear program. The difference between the two figures likely demonstrates different solution approaches or iterations in solving the problem, possibly illustrating the process of finding the maximum flow.

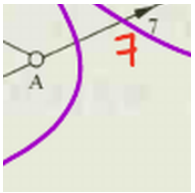
Figures



(a) A directed graph showing a network flow problem. Nodes represent cities (SF, D, H, A, C, NYC), and edges represent flight routes with capacities indicated by numbers on the edges. Arrows indicate direction. A handwritten v_{ij} is above an edge from SF to D, and red arrows indicate a possible flow assignment.



(b) A second directed graph showing a network flow problem. Nodes represent cities (SF, D, H, A, C, NYC), and edges represent flight routes with capacities indicated by numbers on the edges. Arrows indicate direction. Red numbers are written on some edges, indicating a possible flow assignment. The text "Flow = 9" is handwritten to the left. A purple oval encloses a subset of nodes.



(c) Handwritten text "[C]p.368".

Slide 8

Content

Proof of Max Flow = Min Cut
- Let $\{x_{ij}^*\}$ be a (optimal) max flow
- Let C^* be those nodes that some more flow can be pushed from s.
- $t \notin C^*$ (By default, $s \in C^*$)

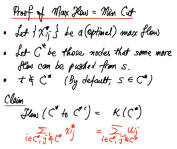
Claim
Flow (C^* to C^{*c}) = $k(C^*)$

$$\sum_{i \in C^*, j \notin C^*} x_{ij}^* = \sum_{i \in C^*, j \notin C^*} u_{ij}$$

Description

Slide 8 of 17 in a Linear Programming course presents a handwritten proof of the Max-Flow Min-Cut theorem. The proof begins by defining an optimal max flow denoted as $\{x_{ij}^*\}$. A set C^* is introduced, representing nodes from which additional flow can be pushed from the source node s. It's noted that the sink node t is not in C^*, and by default, the source node s is in C^*. The core of the proof is a claim stating that the flow from the set C^* to its complement C^{*c} (nodes not in C^*) is equal to the capacity of the cut defined by C^*. This is mathematically expressed as: $\sum_{i \in C^*, j \notin C^*} x_{ij}^* = \sum_{i \in C^*, j \notin C^*} u_{ij}$, where x_{ij}^* represents the flow from node i to node j in the optimal flow, and u_{ij} represents the capacity of the edge from node i to node j. The underlining emphasizes the "Claim" statement. The context of linear programming is crucial here because the Max-Flow Min-Cut theorem is a fundamental result in network flow problems, which are often solved using linear programming techniques. The proof likely continues on subsequent slides.

Figures



(a) Handwritten notes outlining the proof of the Max-Flow Min-Cut theorem. The proof starts by defining an optimal max flow and a set of nodes C* from which additional flow can be pushed. A claim is made that the flow from C* to its complement is equal to the capacity of the cut defined by C*.

Slide 9

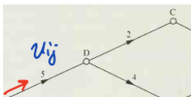
Content

Figure 22.1

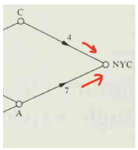
Description

Slide 9 of 17 in a Linear Programming course presents two figures (Figure 22.1, appearing twice) illustrating different flow assignments in the same network flow problem. Both figures depict a directed graph representing a network with nodes representing cities (SF, Denver, Houston, Atlanta, Chicago, NYC) and edges representing flight routes with capacities indicated by numbers on the edges. The first Figure 22.1 shows a possible flow assignment with values v_{ij} indicated by handwritten arrows. The second Figure 22.1 shows a different flow assignment with values indicated by handwritten red numbers on the edges. The text "Flow = 9" is handwritten next to the second Figure 22.1, suggesting the total flow in that assignment is 9. Purple ovals enclose nodes SF, D, and H in the second Figure 22.1, possibly highlighting a subset of nodes relevant to the flow analysis. The page number "[C]p.368" is handwritten, likely referencing a textbook or other source material. The context of linear programming is crucial because these figures illustrate different feasible solutions to a network flow problem, which can be formulated and solved as a linear program. The difference between the two figures likely demonstrates different solution approaches or iterations in solving the problem, possibly illustrating the process of finding the maximum flow or a cut in the network.

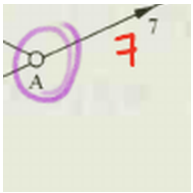
Figures



(a) A directed graph showing a network flow problem. Nodes represent cities (SF, D, H, A, C, NYC), and edges represent flight routes with capacities indicated by numbers on the edges. Arrows indicate direction. A handwritten v_{ij} is above an edge from SF to D, and red arrows indicate a possible flow assignment.



(b) A second directed graph showing a network flow problem. Nodes represent cities (SF, D, H, A, C, NYC), and edges represent flight routes with capacities indicated by numbers on the edges. Arrows indicate direction. Red numbers are written on some edges, indicating a possible flow assignment. The text "Flow = 9" is handwritten to the left. Purple ovals enclose nodes SF, D, and H.



(c) Handwritten text "[C]p.368".

Slide 10

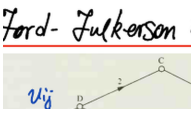
Content

Ford-Fulkerson Algorithm (Augmented Path)

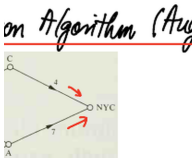
Description

Slide 10 of 17 in a Linear Programming course illustrates the Ford-Fulkerson algorithm using augmented paths. Two figures depict the same network flow problem, representing a network with nodes as cities (SF, Denver (D), Houston (H), Atlanta (A), Chicago (C), New York City (NYC)) and edges as flight routes with capacities shown numerically. The first figure shows an initial flow assignment with handwritten v_{ij} indicating flow values and red arrows showing the flow direction. The second figure (Figure 22.2) shows a subsequent flow assignment after an augmentation step, with flow values indicated by red numbers on the edges. The highlighted path in the second figure represents an augmented path used to increase the overall flow. The red circle around NYC in the second figure emphasizes the sink node. The context of linear programming is crucial because the Ford-Fulkerson algorithm is a method for solving the maximum flow problem, a classic problem in network optimization that can be formulated and solved using linear programming techniques. The two figures illustrate the iterative nature of the algorithm, showing how the flow is increased by finding and augmenting along paths from source to sink.

Figures



(a) A directed graph showing a network flow problem. Nodes represent cities (SF, D, H, A, C, NYC), and edges represent flight routes with capacities indicated by numbers on the edges. Arrows indicate direction. A handwritten v_{ij} is above an edge from SF to D, and red arrows indicate a possible flow assignment.



(b) A second directed graph showing a network flow problem. Nodes represent cities (SF, D, H, A, C, NYC), and edges represent flight routes with capacities indicated by numbers on the edges. Arrows indicate direction. Red numbers are written on some edges, indicating a possible flow assignment. The text "Figure 22.2" is printed below the graph. A red circle encloses the NYC node.

Slide 11

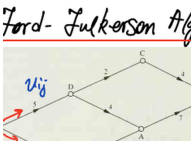
Content

Ford-Fulkerson Algorithm (Augmented Path)
[CJP. 374

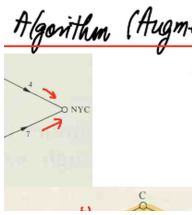
Description

Slide 11 of 17 in a Linear Programming course illustrates the Ford-Fulkerson algorithm using an augmented path. Two figures depict the same network flow problem, with nodes representing cities (SF, Denver (D), Houston (H), Atlanta (A), Chicago (C), New York City (NYC)) and edges representing flight routes with capacities. The first figure shows an initial flow assignment with handwritten v_{ij} and red arrows. The second figure (Figure 22.2) shows the network after an augmentation step, with updated flow values (red numbers) and an augmented path highlighted by orange shading. The red circle around NYC emphasizes the sink node. The text "[CJP. 374]" likely refers to a page number in a cited textbook. The context of linear programming is crucial because the Ford-Fulkerson algorithm is a method for solving the maximum flow problem, solvable using linear programming techniques. The figures illustrate an iteration of the algorithm, showing how flow is increased by finding and augmenting along a path from source to sink.

Figures



(a) A directed graph showing a network flow problem with nodes representing cities (SF, D, H, A, C, NYC) and edges representing flight routes with capacities indicated by numbers on the edges. Arrows indicate direction. A handwritten v_{ij} is above an edge from SF to D, and red arrows indicate a possible flow assignment.



(b) A second directed graph showing a network flow problem with nodes representing cities (SF, D, H, A, C, NYC) and edges representing flight routes with capacities indicated by numbers on the edges. Arrows indicate direction. Red numbers are written on some edges, indicating a possible flow assignment. The text "Figure 22.2" is printed below the graph. A red circle encloses the NYC node. Orange shading highlights an augmented path.

Slide 12

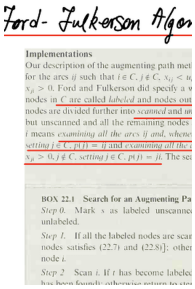
Content

Ford-Fulkerson Algorithm (Augmented Path) Implementations
Our description of the augmenting path method does not specify a way of searching for the arcs ij such that $i \in C, j \notin C, x_{ij} < u_{ij}$, and the arcs ji such that $j \in C, i \notin C$, $x_{ji} > 0$. Ford and Fulkerson did specify a way of doing that. In their terminology, nodes in C are called labeled and nodes outside C are called unlabeled; the labeled nodes are divided further into scanned and unscanned. Initially, the source s is labeled but unscanned and all the remaining nodes are unlabeled. Scanning a labeled node i means examining all the arcs ij and, whenever such an arc satisfies $x_{ij} < u_{ij}, i \in C$, setting $j \in C, p(j) = ij$ and examining all the arcs ji and, whenever such an arc satisfies $x_{ji} > 0, j \notin C$, setting $j \in C, p(j) = ji$. The search may be described as in Box 22.1.
BOX 22.1 Search for an Augmenting Path
Step 0. Mark s as labeled unscanned; mark the remaining nodes as unlabeled.
Step 1. If all the labeled nodes are scanned then stop [the set C of labeled nodes satisfies (22.7) and (22.8)]; otherwise, choose a labeled unscanned node i.
Step 2. Scan i. If t has become labeled then stop (an x-augmenting path has been found); otherwise return to step 1.

Description

Slide 12 of 17 in a Linear Programming course describes the implementation details of the Ford-Fulkerson algorithm for finding augmenting paths in a network flow problem. The slide focuses on the algorithm's search procedure, explaining how it systematically explores the network to find a path from the source to the sink along which flow can be increased. The algorithm uses the concepts of labeled and unlabeled nodes, and scanned and unscanned nodes to manage the search process. The notation x_{ij} represents the flow on arc (i, j), and u_{ij} represents the capacity of arc (i, j). The set C represents the set of labeled nodes. The algorithm's steps are clearly outlined in "BOX 22.1," providing a step-by-step guide to the search for an augmenting path. The context of linear programming is crucial because the Ford-Fulkerson algorithm is a fundamental method for solving the maximum flow problem, a classic problem in network optimization that can be formulated and solved using linear programming techniques. The slide provides a detailed explanation of the algorithm's mechanics, which is essential for understanding how it efficiently finds the maximum flow in a network.

Figures



(a) Handwritten title "Ford-Fulkerson Algorithm (Augmented Path)" and a description of the Ford-Fulkerson algorithm's implementation, including a boxed section labeled "BOX 22.1 Search for an Augmenting Path" with steps 0, 1, and 2 detailing the algorithm's search process. The text describes labeled and unlabeled nodes, scanned and unscanned nodes, and the conditions for adding nodes to the set C.

Slide 13

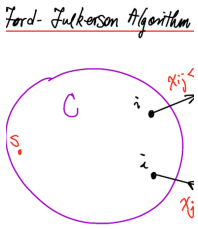
Content

Ford-Fulkerson Algorithm (Augmented Path)

Description

Slide 13 of 17 in a Linear Programming course illustrates a step in the Ford-Fulkerson algorithm for finding augmenting paths. The diagram shows a network represented by nodes and directed edges. A set of nodes, denoted by C (enclosed in a purple circle), represents the set of labeled nodes in the algorithm. Nodes outside the circle are unlabeled. The arrows represent edges in the network, with labels indicating the flow (x_{ij}) and capacity (u_{ij}) conditions. The red labels "$x_{ij} < u_{ij}$" and "$x_{ji} > 0$" on the arrows indicate the conditions for selecting edges to extend the augmenting path. The nodes labeled "j" outside the circle represent potential nodes to add to the set C during the search for an augmenting path. The nodes "s" and "t" likely represent the source and sink nodes, respectively. The dashed circles around the "j" nodes highlight their status as potential additions to the set C. The context of linear programming is crucial because the Ford-Fulkerson algorithm is a method for solving the maximum flow problem, a classic problem in network optimization that can be formulated and solved using linear programming techniques. The diagram visually explains how the algorithm searches for an augmenting path by examining edges based on flow and capacity constraints.

Figures



(a) A hand-drawn diagram illustrating a step in the Ford-Fulkerson algorithm. A purple circle labeled "C" contains several nodes, including a node labeled "i" and another labeled "i". Outside the circle are nodes labeled "j" and "j". Arrows connect nodes within and outside the circle. An arrow from a node labeled "i" to a node labeled "j" is labeled with "$x_{ij} < u_{ij}$" in red. Another arrow from a node labeled "i" to a node labeled "j" is labeled with "$x_{ji} > 0$" in red. A small red dot is labeled "s". A small red dot is labeled "t". The nodes labeled "j" are enclosed in dashed purple circles.

Slide 14

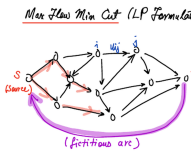
Content

Max Flow Min Cut (LP Formulation)

Description

Slide 14 of 17 in a Linear Programming course illustrates the Max Flow Min Cut theorem and its linear programming formulation. The diagram depicts a network flow problem, where nodes represent points in a network and directed edges represent connections with capacities. The source node "S" and sink node "t" are clearly marked. The weights w_{ij} on some edges likely represent capacities. A crucial element is the addition of a "fictitious arc" connecting the source and sink, with cost $c_{ts} = -1$ and infinite capacity $u_{ts} = +\infty$. This fictitious arc is a key component of the linear programming formulation of the Max Flow Min Cut theorem, allowing the problem to be expressed as a linear program. The variables x_{ts} likely represent the flow along this fictitious arc. The context of linear programming is essential because this slide demonstrates how a combinatorial optimization problem (finding the maximum flow) can be formulated as a linear program, which can then be solved using linear programming techniques. The diagram visually represents the network structure, while the text provides the parameters for the linear programming formulation.

Figures



(a) A hand-drawn diagram illustrating a network flow problem. The diagram shows a network with nodes and directed edges. The nodes are labeled with circles, and some nodes are labeled with additional text, such as "S (source)" and "t (sink)". The edges are labeled with arrows indicating direction and some edges have labels like "w_{ij}". A purple arc connects the source and sink nodes, labeled "(fictitious arc)". The text "x_{ts}, $c_{ts} = -1$, $u_{ts} = +\infty$" is written below the diagram.

Slide 15

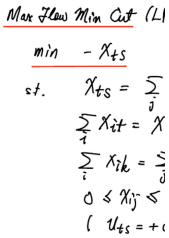
Content

Max Flow Min Cut (LP Formulation)
min -x$_{ts}$
s.t.
$x_{ts} = \sum_j x_{sj}$
$\sum_i x_{it} = x_{ts}$
$\sum_i x_{ik} = \sum_j x_{kj} \quad k \neq s, t$
$0 \leq x_{ij} \leq u_{ij}$
$(u_{ts} = +\infty)$

Description

Slide 15 of 17 in a Linear Programming course presents the linear programming (LP) formulation of the Max Flow Min Cut problem. The objective function is to minimize $-x_{ts}$, where x_{ts} represents the flow on a fictitious arc from the sink to the source. Minimizing $-x_{ts}$ is equivalent to maximizing x_{ts}, which represents the total flow through the network. The constraints ensure flow conservation at each node: the total flow entering a node equals the total flow leaving the node. The summation notation $\sum_j x_{sj}$ represents the total flow leaving the source node, and $\sum_i x_{it}$ represents the total flow entering the sink node. The constraint $\sum_i x_{ik} = \sum_j x_{kj}$ for $k \neq s, t$ enforces flow conservation at all intermediate nodes. The inequalities $0 \leq x_{ij} \leq u_{ij}$ represent the capacity constraints on each arc (i, j), where u_{ij} is the capacity of the arc. The capacity of the fictitious arc from the sink to the source is assumed to be infinite $(u_{ts} = +\infty)$. This LP formulation provides a way to solve the maximum flow problem using linear programming techniques. The context of linear programming is crucial because this slide shows how a network flow problem can be transformed into an equivalent linear program, which can then be solved using standard LP algorithms.

Figures



(a) A handwritten linear program formulation for the Max Flow Min Cut problem. The objective function is to minimize $-x_{ts}$, where x_{ts} represents the flow from the source to the sink through a fictitious arc. The constraints ensure flow conservation at each node (except the source and sink) and respect the capacity constraints u_{ij} on each arc (i, j). The capacity of the fictitious arc is assumed to be infinite ($u_{ts} = +\infty$).

Slide 16

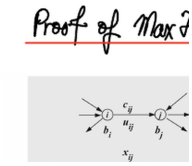
Content

Proof of Max Flow = Min Cut (LP)

Description

Slide 16 of 17 in a Linear Programming course demonstrates a proof of the Max Flow Min Cut theorem within the context of linear programming. The figure illustrates a key step in the proof: handling upper bounds on arc capacities. The top part of the figure shows a single arc (i, j) with capacity u_{ij} and flow x_{ij}. The bottom part shows the same arc decomposed by introducing a new node k, splitting the arc into two arcs: (i, k) with capacity u_{ij} and flow x_{ij}, and (k, j) with infinite capacity and flow t_{ij}. The equation $x_{ij} + t_{ij} = u_{ij}$ at node k ensures flow conservation. The equations to the right of the figure show the complementary slackness conditions derived from the linear programming dual. These conditions relate the flow (x_{ij}) on an arc to the dual variables (y_i, y_j) and the cost (c_{ij}) associated with that arc. The conditions are based on whether the flow is zero, at capacity, or strictly between zero and capacity. The final equation, $x_{ij} + t_{ij} = u_{ij}$, explicitly shows how the introduction of the new node k allows the handling of the upper bound u_{ij} on the flow. The context of linear programming is crucial because this slide demonstrates a proof technique that leverages the duality theory of linear programming to prove a fundamental theorem in network flow optimization.

Figures



(a) A diagram showing a transformation of a network flow graph. The top half shows a simple edge between nodes i and j with capacity u_{ij} and flow x_{ij}. The bottom half shows the same edge split into two edges by adding a new node k. The edge from i to k has capacity u_{ij} and flow x_{ij}. The edge from k to j has capacity ∞ and flow t_{ij}. The node k has flow conservation $x_{ij} + t_{ij} = u_{ij}$.

Slide 17

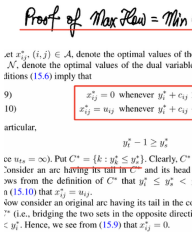
Content

Proof of Max Flow = Min Cut (LP)
Let $x_{ij}^*, (i,j) \in A,$ denote the optimal values of the primal variables, and let y_i^*, $i \in N,$ denote the optimal values of the dual variables. Then the complementarity conditions (15.6) imply that
(15.9) $x_{ij}^* = 0$ whenever $y_i^* + c_{ij} > y_j^*$
(15.10) $x_{ij}^* = u_{ij}$ whenever $y_i^* + c_{ij} < y_j^*$
In particular,
$y_i^* - 1 \geq y_s^*$
(since $u_{ts} = \infty$). Put $C^* = \{k : y_k^* \leq y_s^*\}$. Clearly, C^* is a cut.
Consider an arc having its tail in C^* and its head in the complement of C^*. It follows from the definition of C^* that $y_i^* \leq y_s^* < y_j^*$. Since c_{ij} is zero, we see from (15.10) that $x_{ij}^* = u_{ij}$.
Now consider an original arc having its tail in the complement of C^* and its head in C^* (i.e., bridging the two sets in the opposite direction). It follows then that $y_j^* \leq y_s^* < y_i^*$. Hence, we see from (15.9) that $x_{ij}^* = 0$.

Description

Slide 17 of 17 in a Linear Programming course completes the proof of the Max Flow Min Cut theorem using linear programming duality. The proof starts by stating the complementarity conditions (15.9) and (15.10) which relate the optimal primal variables (x_{ij}^* representing flow) and dual variables (y_i^*) of the linear programming formulation of the max-flow problem. A cut C^* is defined as the set of nodes k such that $y_k^* \leq y_s^*$, where s is the source node. The proof then considers two cases: arcs with tails in C^* and heads outside C^*, and arcs with tails outside C^* and heads in C^*. By applying the complementarity conditions (15.9) and (15.10) and the definition of the cut C^*, the proof shows that the flow on arcs crossing the cut C^* in one direction is at capacity, while the flow on arcs crossing in the opposite direction is zero. This directly implies that the maximum flow equals the capacity of the minimum cut, thus proving the Max Flow Min Cut theorem. The context of linear programming is crucial because this slide demonstrates a proof technique that leverages the duality theory of linear programming to prove a fundamental theorem in network flow optimization. The notation u_{ij} represents the capacity of arc (i,j), and c_{ij} represents the cost associated with that arc (likely 0 in this context).

Figures



(a) Handwritten proof of the Max Flow Min Cut theorem using linear programming duality. The proof uses complementarity conditions (15.9) and (15.10) derived from the linear program and its dual. A cut C^* is defined based on the dual variables y_i^*. The proof then analyzes arcs with tails and heads in C^* and its complement to show that the maximum flow equals the minimum cut.